Group 2

FrameRoom
Software Development Plan (Small Project)
Version 1.0

Revision History

Date	Version	Description	Author
13/10/2026	1.0	Initial draft of the Software Development Plan for FrameRoom.	K. Nguyen

Group 2

Table of Contents

1. Introduction **3**

1.1 Purpose *3*

1.2 Scope *3*

1.3 Overview *3*

2. Project Overview **3**

2.1 Project Purpose, Scope, and Objectives *3*

2.2 Assumptions and Constraints *4*

2.3 Project Deliverables *4*

3. Project Organization **5**

3.1 Organizational Structure *5*

3.2 Roles and Responsibilities *5*

4. Management Process **6**

4.1 Project Estimates *6*

4.2 Project Plan *6*

4.2.1 Phase Plan *6*

4.2.2 Iteration Objectives *7*

4.2.3 Releases *8*

4.2.4 Project Schedule *9*

4.2.5 Project Resourcing *11*

4.3 Project Monitoring and Control *12*

4.3.1 Requirements Management *12*

4.3.2 Reporting and Measurement *12*

4.3.3 Risk Management *12*

4.3.4 Configuration Management *14*

Group 2

Software Development Plan (Small Project)

1. Introduction

1.1 Purpose

The purpose of this Software Development Plan is to gather all information necessary to control the FrameRoom project. It describes the approach to the development of the platform (including the web-based application and AI integrations) and serves as the top-level plan used by managers to direct the development effort.

The following people use the Software Development Plan:

- **The Project Manager** uses it to plan the project schedule, allocate resources, and track progress against the timeline.
- **Project Team Members** (frontend/backend developers, AI engineers, UI/UX designers) use it to understand what tasks they need to do, deadlines, and project dependencies.

1.2 Scope

This Software Development Plan describes the overall plan to be used by the FrameRoom project, including the deployment of the web-based product and preparation for a future desktop application. The details of individual sprints or iterations will be described in the Iteration Plans.

The plans outlined in this document are strictly based upon the product requirements defined in the FrameRoom Vision Document, specifically focusing on the photography workflow management features and AI components.

1.3 Overview

This Software Development Plan contains the following information:

- **Project Overview:** Provides a description of the project's purpose, scope, and objectives, as well as the expected project deliverables.
- **Project Organization:** Describes the organizational structure of the project team and individual roles.
- **Management Process:** Outlines project estimates, project plans (schedule and resources), and methods for project monitoring and control (including risk and configuration management).

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Purpose: The primary purpose of FrameRoom is to eliminate visual communication barriers and streamline the fragmented collaboration workflow in professional photography-from client discovery and brief definition to photo proofing and final delivery.

Scope: The project encompasses a web-based platform featuring an Smart AI Search, a centralized Shoot Room workspace, automated timeline/SLA tracking, and a workflow-backed trust review system. It focuses on web deployment first, with architectural preparation for future desktop packaging via Tauri.

Objectives:

- Deliver a fully functional web platform covering the end-to-end photography workflow.
- Integrate a reliable hybrid search engine (FastAPI + PostgreSQL pgvector) and lightweight CV/NLP capabilities.
- Achieve clear role-based access control and workspace isolation for photographers, clients, and system administrators.

2.2 Assumptions and Constraints

Assumptions:

- Team members possess sufficient technical proficiency in web development (FastAPI, modern JavaScript frontend frameworks) and database systems.
- Target users (photographers, clients, studios) have access to modern web browsers and stable internet connections.

Constraints:

- Schedule Constraint: Fixed academic timeline requiring strict adherence to milestone delivery dates.
- Budget Constraint: Zero or minimal infrastructure budget, relying on local development environments, open-source tools, and free-tier cloud hosting.
- Resource Constraint: Small development team size, requiring careful prioritization of core MVP features over non-essential extensions.

2.3 Project Deliverables

Artifact / Deliverable	Description	Target Delivery Date
Vision Document	Defines high-level goals, problem statements, stakeholders and product features. First version in Inception; revised in Elaboration.	Sprint 1 (PA1, 17/10/2026); revised 13/11/2026
Software Development Plan	Schedule, resources, risk management and team roles. First version in Inception; revised after feedback.	Sprint 1 (PA1, 17/10/2026); revised 30/10/2026
Software Requirements Specification (SRS)	Provides detailed functional and non-functional requirement specifications and use cases.	Sprint 2 (30/10/2026); revised 13/11/2026
Architecture & System Design Specs	UI prototypes, Software Architecture Document, ERD (pgvector design) and API contract.	Sprint 3 (13/11/2026)
Test Plan	Test strategy, scope, environments and responsibilities.	Sprint 3 (13/11/2026)
Prototype	Working prototype of the core architecture (Build 2).	Sprint 3 (13/11/2026)
Test Cases	Test cases derived from the use cases.	Sprint 4 (27/11/2026)

Source Code & Executables	Working web platform: Astryx frontend, Fastify API, FastAPI AI service, Supabase migrations (Build 4).	Sprint 5 (12/12/2026)
Test Report & Defect List	Test execution results, bug log, validation against requirements.	Sprint 5 (12/12/2026)
Final Presentation & Demo	Presentation and live demo at the final review.	Week 10 (Sprint 5)
Final Submission	All revised artifacts packaged for submission.	12/12/2026

3. Project Organization

3.1 Organizational Structure

The team has five members and a flat structure. The Team Lead coordinates planning, meetings and reporting, but every member owns at least one module and reviews other members' work. Each role has one main person and one backup.

Review authorities and mechanisms:

- The course instructor and teaching assistants (TAs) review the plan, the Vision document and weekly reports at each milestone.
- Every pull request needs at least one reviewer and a passing CI pipeline before merging. The reviewer must not be the author.
- The team holds a weekly scrum on Discord answering three questions: what was done since last week, what issues the team faces, and what will be accomplished by the next scrum.
- Tasks are planned in GanttProject (overall schedule) and tracked in JIRA (project key FR) per sprint.

3.2 Roles and Responsibilities

Person	Project Role	Backup For	Responsibilities
Nguyen Dang Khoi Nguyen	Team Lead / Project Manager; DevOps	Backend; Tester	Writes and updates the project plan and assigns tasks; monitors progress and writes the weekly reports; coordinates meetings; acts as the contact point for the project; sets up and maintains the development environment and the CI/CD pipeline
Dao Thi Quynh Anh	Tester; Implementer (AI Search/CV)	Project Manager; Business Analyst	Writes the test plan and test cases, performs system tests and reports defects; implements the AI Search service and its data pipeline, and evaluates search quality

Phan Le Dang Khoa	Business Analyst; Designer (architecture and UI); Implementer (Frontend)	Project Manager	Documents requirements and use cases, communicates them to the team and coordinates requirement reviews; designs the user interface and prototypes and writes the Software Architecture Document; implements the frontend, performs unit tests and reviews source code.
Ha Thao Ly	Database designer; Implementer (database migrations/RLS)	Tester	Designs the database (ERD, schema and access policies) and implements database migrations; prepares the demo data; performs unit tests and reviews source code.
Le Quoc Khai	Implementer (Backend)	Database	Implements the backend API according to the API contract; performs unit tests and reviews source code.

4. Management Process

4.1 Project Estimates

4.2 Project Plan

4.2.1 Phase Plan

Phase	Sprint	Weeks	Dates	Assignment	Focus
Inception	Sprint 1	1-2	03/10 - 16/10/2026	PA0, PA1	Project proposal and initial requirements: scope, plan, Vision, tools, first design artifacts
Elaboration	Sprint 2	3-4	18/10 - 30/10/2026	PA2	Detailed requirements and initial design: use cases, architecture draft, first prototype
Elaboration	Sprint 3	5-6	31/10 - 13/11/2026	PA3	Detailed design and test planning; architecture proven by a prototype
Construction	Sprint 4	7-8	14/11 - 27/11/2026	PA4	Implementation and testing of the main flow
Construction	Sprint 5	9-10	28/11 - 12/12/2026	PA5	Implementation and testing; final review, demo and submission

Milestones and achievement criteria:

Milestone	Week	Achievement criteria
M1 - Lifecycle Objectives	2	Vision and project plan submitted; scope of the core and optional features agreed; roles, tools and repository ready; ERD v1 and API contract v1 drafted.
M2 - Architecture baseline	6	Use case document, Design document (UI prototype and architecture) and test plan submitted; prototype demonstrates login, role-based access, portfolio and first AI Search.
M3 - Initial operational capability	8	Main flow works end to end: search, Shoot Brief, booking and Shoot Room messaging.
M4 - Final release	10	All core features complete; test report and defect list delivered; presentation and demo done; final submission made.

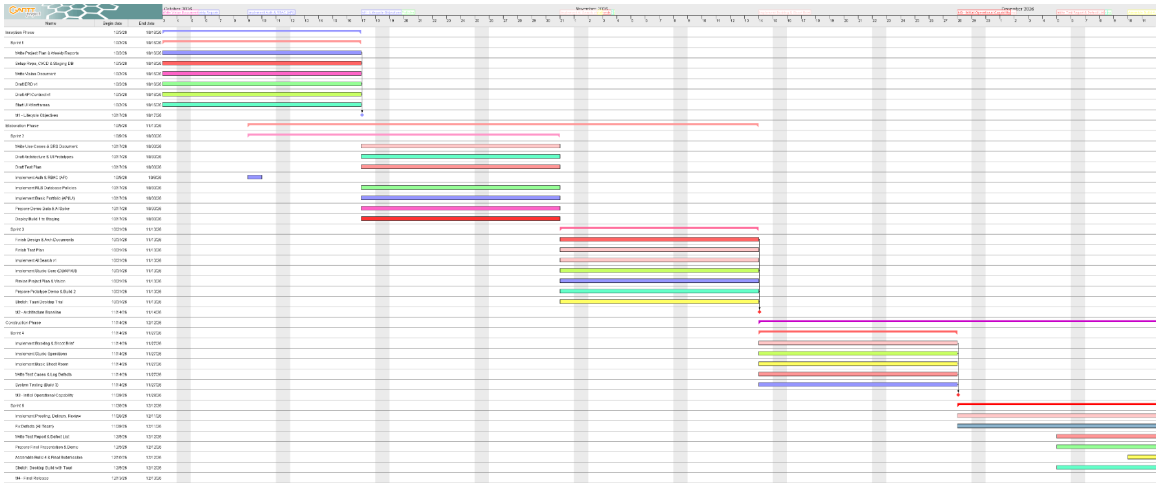


Figure 4.1: FrameRoom Project Schedule and Work Breakdown Structure (Gantt Chart)

Schedule Summary & Critical Path Note:

Total Duration: 10 weeks (5 Sprints, from October 03, 2026 to December 12, 2026).

- **Critical Path:** Inception & Document Baseline (Sprint 1) => Core Architecture & Authentication (Sprint 2) => AI Search & Studio Core Prototype (Sprint 3 / Milestone M2) => Booking & Shoot Room Workflows (Sprint 4) => Photo Proofing, Final Testing & Defect Fixing (Sprint 5 / Milestone M4).
- **Stretch Goals**: Desktop version packaging via Tauri is strictly set as an optional task in Sprint 3 (spike) and Sprint 5 (build) to ensure zero impact on core web deliverables if schedule delays occur.

Work breakdown (summary): project management and documentation; requirements and design; database and Supabase; backend API; frontend; AI Search; testing; DevOps and CI/CD.

4.2.2 Iteration Objectives

Sprint	Objectives
Sprint 1 (Inception, weeks 1-2)	Write the project plan and Vision document; identify risks; assign roles and select tools (Discord, GanttProject, JIRA, GitHub); set up the repository, CI and the shared Supabase Cloud staging project; draft ERD v1 and API contract v1; start wireframes; submit two weekly reports.
Sprint 2 (Elaboration, weeks 3-4)	Write the use case document and SRS; draft architecture and UI prototypes; draft the test plan; implement authentication, role-based access (RBAC) and RLS, and basic Portfolio; run the demo data pipeline; run a technical spike on pgvector.
Sprint 3 (Elaboration, weeks 5-6)	Finish the Design document, architecture document and test plan; implement AI Search v1 (NLP and CV, hybrid ranking) and Studio core (profile, services, members); demo the prototype at the end of week 6. Stretch: a one to two day trial of Tauri, a framework that packages a web application as a desktop application, to check that login and image upload still work in the desktop version.
Sprint 4 (Construction, weeks 7-8)	Implement Booking with Shoot Brief, studio operations (bookings, team), and basic Shoot Room (messages, image attachments); write test cases; start recording defects.
Sprint 5 (Construction, weeks 9-10)	Implement photo selection (proofing), delivery and review; optional features if capacity allows; fix defects; deliver test report, final presentation and demo; final submission. Stretch: build the desktop version with Tauri, only if the Sprint 3 trial succeeded and all core features are stable.

Priorities: core features are built first. Optional features and the desktop version are stretch goals and are the first items to be deferred if the schedule slips.

4.2.3 Releases

The team plans four builds for testing.

Build	Date (end of)	Type	Content
Build 1	Sprint 2 (30/10/2026)	Internal alpha	Walking skeleton: login, RBAC/RLS, portfolio upload and view, deployed on staging.
Build 2	Sprint 3 (13/11/2026)	Prototype demo	Build 1 plus AI Search v1 and Studio core;

			demonstrated to instructor/TAs.
Build 3	Sprint 4 (27/11/2026)	Beta	Main flow: search, Shoot Brief, booking, Shoot Room; system testing by the Tester.
Build 4	Sprint 5 (12/12/2026)	Release candidate / final	All core features, defects fixed, used for final demo and submission.

Sprint 1 produces no build because it focuses on documents and environment setup. A Tauri desktop build is a stretch item and is not counted among the four builds.

4.2.4 Project Schedule

Iteration / Milestone	**Start**	**End**	**Key deliverables**
Sprint 1 - Inception	03/10/2026	16/10/2026	Project plan, Vision document, weekly reports 1 and 2, ERD v1, API contract v1
Sprint 2 - Elaboration	17/10/2026	30/10/2026	Use case document, design drafts, Build 1
Sprint 3 - Elaboration	31/10/2026	13/11/2026	Revised plan and Vision, Design document, test plan, prototype, Build 2
Sprint 4 - Construction	14/11/2026	27/11/2026	Test cases, source code, Build 3
Sprint 5 - Construction	28/11/2026	12/12/2026	Test report, defects list, presentation, demo, Build 4, final submission

Main tasks of each sprint and the team members responsible:

Sprint	**Main task**	**Responsible**	**Due**
Sprint 1	Write the project plan and weekly reports 1 and 2	Khoi Nguyen	16/10/2026
Sprint 1	Write the Vision document	Quynh Anh	16/10/2026
Sprint 1	Set up the repository, CI pipeline and shared cloud database	Khoi Nguyen	16/10/2026
Sprint 1	Draft ERD v1	Ha Thao Ly	16/10/2026
Sprint 1	Draft API contract v1	Le Quoc Khai, Dang Khoa	16/10/2026

Group 2

Sprint 1	Start UI wireframes	Dang Khoa	16/10/2026
Sprint 2	Write the use case document and SRS	Quynh Anh	30/10/2026
Sprint 2	Draft architecture and UI prototypes	Dang Khoa	30/10/2026
Sprint 2	Draft the test plan	Quynh Anh	30/10/2026
Sprint 2	Implement authentication and role-based access (API)	Le Quoc Khai	30/10/2026
Sprint 2	Implement database access policies	Ha Thao Ly	30/10/2026
Sprint 2	Implement basic portfolio (API and UI)	Le Quoc Khai, Dang Khoa	30/10/2026
Sprint 2	Prepare demo data and run the technical trial of AI Search	Quynh Anh	30/10/2026
Sprint 2	Deploy Build 1 to staging	Khoi Nguyen	30/10/2026
Sprint 3	Finish the Design document and architecture document	Quynh Anh	13/11/2026
Sprint 3	Finish the test plan	Quynh Anh	13/11/2026
Sprint 3	Implement AI Search v1	Quynh Anh	13/11/2026
Sprint 3	Implement studio core (database, API and UI)	Ha Thao Ly, Le Quoc Khai, Dang Khoa	13/11/2026
Sprint 3	Revise the project plan and Vision document	Khoi Nguyen	13/11/2026
Sprint 3	Prepare the prototype demo and Build 2	Khoi Nguyen	13/11/2026

Sprint 3	Stretch: trial of Tauri (check login and image upload in the desktop version)	Khoi Nguyen, Dang Khoa	13/11/2026
Sprint 4	Implement booking with Shoot Brief (database, API and UI)	Ha Thao Ly, Le Quoc Khai, Dang Khoa	27/11/2026
Sprint 4	Implement studio operations (bookings, team)	Le Quoc Khai, Dang Khoa	27/11/2026
Sprint 4	Implement basic Shoot Room	Le Quoc Khai, Dang Khoa	27/11/2026
Sprint 4	Write test cases and record defects	Quynh Anh, Khoi Nguyen	27/11/2026
Sprint 4	System testing of Build 3	Quynh Anh, Khoi Nguyen	27/11/2026
Sprint 5	Implement photo selection, delivery and review	Ha Thao Ly, Le Quoc Khai, Dang Khoa	12/12/2026
Sprint 5	Fix defects	All implementers	12/12/2026
Sprint 5	Write the test report and defect list	Quynh Anh	12/12/2026
Sprint 5	Prepare the final presentation and demo	Khoi Nguyen	12/12/2026
Sprint 5	Assemble Build 4 and the final submission	Khoi Nguyen	12/12/2026
Sprint 5	Stretch: build the desktop version with Tauri, only if the Sprint 3 trial succeeded	Khoi Nguyen, Dang Khoa	12/12/2026

4.2.5 Project Resourcing

Skill	**Primary**	**Phases where needed**
Project management and DevOps	Khoi Nguyen	All phases

Requirements, UI design, React/Astryx frontend	Dang Khoa	Inception to Construction
PostgreSQL, RLS	Ha Thao Ly	Inception to Construction
Node.js/Fastify backend	Le Quoc Khai	Mainly Sprints 2-5
Python/FastAPI, pgvector, system testing	Quynh Anh	Mainly Sprints 2-3 for AI Search; testing is heaviest in Sprints 4-5

Tools: Discord (communication), GanttProject (schedule), JIRA (sprint tasks), GitHub and GitHub Actions (version control, CI/CD), Supabase Cloud (shared database, auth, storage), Figma (UI), AI coding assistants.

Training needs (self-study with short knowledge-sharing sessions):

Topic	**Who**	**Target date**
Supabase RLS and Auth	Ly, Khai, Khoa	End of Sprint 1
Astryx design system	Khoa, backup: Nguyen	Week 3
CV and NLP hybrid search	Quynh Anh, backup: Nguyen	Week 4
Tauri and Rust basics (stretch)	Khoa	Before the Tauri spike in Sprint 3

4.3 Project Monitoring and Control

4.3.1 Requirements Management

The requirements for this system are captured in the Vision document and, from Elaboration onward, in the use case document and SRS. Each requirement is classified as a core or an optional feature for the current phase.

Requested changes to requirements are captured as Change Requests, created as JIRA issues of type Change Request. The Business Analyst assesses the impact on scope, schedule and risk, and the Team Lead approves or rejects the request after discussion in the weekly scrum. Approved changes update the Vision document, the plan and the affected JIRA stories. For example, adding a desktop version of the application to the project scope is handled as a Change Request.

4.3.2 Reporting and Measurement

4.3.3 Risk Management

Risks are identified in the Inception phase and reviewed at least once per iteration. The risks of the greatest magnitude are listed first.

Group 2

Risk	Ranking	Description and impact	Mitigation strategy / contingency plan
R1. Scope creep	High	Extras beyond the core scope (for example a desktop version of the application) consume time needed for core features and documents.	Core features first; extras are stretch goals handled by Change Request. Contingency: defer optional items starting from Sprint 5.
R2. RLS errors and data leakage	High	Wrong access policies expose or block data between user roles.	Test every table and policy with each role; the Tester reviews policies; privileged database keys are used only on the server and shared with as few people as possible.
R3. AI Search quality	High	Multilingual CLIP and the hybrid ranking may return poor results, and tuning takes more time than planned.	Early spike in Sprint 2; measure with a small evaluation set; fall back to tag and text search if vector search is not good enough.
R4. Quality and security of AI-generated code	Medium	Generated code may contain bugs, hidden vulnerabilities or code the author does not understand.	Mandatory pull request review, CI (lint, typecheck, test, build), small tickets with acceptance criteria; the author must explain the code.
R5. Shared cloud database problems	Medium	A bad migration or seed on the shared Supabase project breaks everyone's work; free-tier limits on storage or inactivity pausing may block work or the demo.	Migrations only via CI after merge; try risky migrations locally first; separate staging and demo projects; monitor usage and clean test images; reactivate the project before the demo.
R6. Interface mismatch between frontend, backend and AI service	Medium	Contract drift causes integration bugs late in the schedule.	The API contract and shared data schemas are the single source of truth; changes go through pull requests; integration checks at the end of each sprint.

R7. Short schedule	Medium	The project lasts ten weeks, and Sprint 5 contains both feature work and final hardening.	Fixed milestones and builds; limit work in progress; start test cases in Sprint 3; freeze features at the start of week 10.
R8. Image data and licensing	Low	Demo images may have license limits.	Use only images whose licenses allow reuse and record the source and license of each image.
R9. Tauri integration (stretch)	Low	Login redirects, file upload and installer builds in the desktop webview may be harder than expected.	Time-boxed spike in Sprint 3 with a go/no-go decision; drop it if core features are at risk.

4.3.4 Configuration Management

Problems and change requests are recorded as JIRA issues (project key FR). Defects are recorded as Bug issues and requirement changes as Change Requests, as described in section 4.3.1. They are reviewed in the weekly scrum and approved by the Team Lead.

All source code, test scripts, database migrations and project documents are kept in one GitHub repository. Work is done on short-lived branches named after the JIRA issue (for example feature/FR-123-short-description) and merged into the integration branch through a pull request with at least one reviewer and a passing CI pipeline. The main branch holds stable versions only. Each commit message contains the JIRA key.

Documents carry a version number (major.minor) and a revision history table. At the end of each sprint the team creates a baseline by tagging the repository (for example sprint-2) together with the documents and the build of that sprint.

The GitHub repository is the master copy and every member holds a local clone. The database schema is defined by migration files and the demo data by a seed script, so the shared cloud database can be rebuilt from the repository if it is damaged.